

Potato

- **Tidsgräns:** 10 minuter
- **Miljö:** en GPU (≈16 GB VRAM), ingen internetuppkoppling
- **Lösningens storlek:** `solution.ipynb` ≤ 1 MB
- **Lagring:** 5 GB

Uppgift

Din vän föreslår att ni spelar ett gissningsspel. Han, i rollen som domare, väljer ett hemligt ord från ett fast vokabulär, och du måste hitta det inom högst 30 turer. Varje tur jämför domaren två ord och rapporterar vilket som är semantiskt närmast det hemliga ordet. Varje spel startar från det fasta paret `lamp` vs `potato`, eftersom de är två av din väns favoritsaker. Ditt program föreslår sedan ett nytt ord. Vinnaren av jämförelsen behålls och jämförs mot ditt nästa förslag. Du vinner ett spel i samma stund du föreslår exakt det hemliga ordet. Matchningen är skiftlägesokänslig. Varje ord du föreslår måste finnas i `dataset/vocabulary.json`.

Det finns ett fullständigt exempel i `solution.ipynb` med protokoll och datainläsning. Du kan ändra klassen `PublicEmbeddingPlayer`. Ditt program initieras en gång och spelar alla spel i en enda körning; protokollet skapar en ny `PublicEmbeddingPlayer` i början av varje spel.

Domaren

Ditt program skickar ett JSON-objekt till domaren och domaren svarar med ett JSON-objekt.

Ett genomgången exempel, där det hemliga ordet visas endast för att förklara protokollet:

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}
-> {"status": "win"}

<- matches: game won
```

Turerna indexeras från 1 till 30.

Alternativen för `verdict` är `first` som betyder att `word1` är närmare, `second` som betyder att `word2` är närmare eller `same` som betyder att båda orden är lika nära det hemliga ordet.

`winner_word` är det ord som behålls till nästa jämförelse. Vid ett `same`-utfall stannar det första ordet kvar.

Dataset

Delas av alla splittar:

- `dataset/vocabulary.json` — 1602 unika ord med små bokstäver. Det hemliga ordet är alltid ett av dessa.
- `dataset/public_embeddings.npy` — `float32`, form (1602, 2560). Rad `i` motsvarar ord `i` i vokabulären. Dessa är *publika* embeddings; domaren använder en annan, privat representation.

Splittarna är mängder av hemliga ord:

Split	Ord	Svar	Använd den för att
<code>test_public</code>	120	<code>dataset/test_public.json</code>	köra din lösning och poängsätta dig själv
<code>test_leaderboard_a</code>	120	dolda	live-leaderboard
<code>test_leaderboard_b</code>	120	dolda	slutlig rangordning

Det finns ingen `train-split` — inget anpassas utifrån märkta rader.

Tillhandahållna modeller

Två förtränade embeddingmodeller levereras med uppgiften och får användas:

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

Båda måste laddas från sin lokala sökväg; ett Hugging Face hub-id såsom "BAAI/bge-m3" utlöser en nedladdning och misslyckas, eftersom bedömningen sker offline. Varje katalog innehåller en körbar `example.py` som visar det offline-anropet.

Tillgängliga bibliotek: `numpy`, `torch`, `sentence-transformers`. Ingen internetuppkoppling, inga nedladdningar, inga andra paket.

Utdata

Ingen. Detta är en interaktiv uppgift: din lösning skriver ingen svarsfil; den kommunicerar med domaren över stdin/stdout enligt beskrivningen ovan.

Mått

Ett spel som löses på tur t ger $1.0 - 0.02 \times \max(0, t - 10)$; ett spel som inte löses inom 30 turer ger 0. Alltså ger turerna 1–10 1.00, tur 20 ger 0.80, tur 30 ger 0.60.

Din uppgiftspoäng är den genomsnittliga spelpoängen $\times 100$, mellan 0.00 och 100.00.

Tidsgränsen på 10 minuter är en enda budget som täcker uppstart, förberedelser och alla 120 spel i testmängden.

Så här lämnar du in

1. Öppna `solution.ipynb`, redigera `PublicEmbeddingPlayer` och kör alla celler för att kontrollera att det fungerar.
2. Kontrollera det gärna lokalt: `python local_test.py solution.ipynb --limit 5`. Den lokala domaren använder de *publika* embeddingarna, så dess poäng är endast vägledande.
3. Spara `solution.ipynb`.
4. Öppna Git-fliken i JupyterLabs vänstra sidofält.
5. Stage:a `solution.ipynb` (ikonen + intill den).
6. Skriv ett commit-meddelande och klicka på Commit.
7. Klicka på molnet med uppåtpil för att pusha.
8. Återvänd till denna Contest-sida och klicka på Submit, med ett commit-meddelande som matchar det du angav.

Lämna in exakt en fil, med namnet `solution.ipynb`, som täcker alla nödvändiga förberedelser och inferens.